PROJECT REPORT

Submitted by

AARON TOM ALEX

AJC23CS002

S6 CSE A BATCH

02

TITLE: **HANGMAN'S GAME using PYTHON**

INDEX

1.	Abstract	3
2.	Descriotion	4
3.	Libraries & Tools	5
4.	System Design & Workflow	6
5.	Implementation	8
6.	Results & Outputs	9
7.	Conclusion & Scope	11

ABSTRACT

This project features a **2-player, Wild West-themed Hangman game** developed in Python, utilizing a modular, object-oriented architecture. The application leverages **Tkinter** and **BreezyPyGUI** for its graphical interface, integrated with **Pygame** and **PIL (Python Imaging Library)** to deliver a multimedia-rich experience including background music, dynamic image handling, and responsive layout resizing.

The system employs a strict separation of concerns between its **backend logic** and **event-driven GUI**. The core logic module manages word validation, state tracking, and win/loss detection, while the interface provides a thematic "bounty" hunt experience. In this version, Player 1 sets the target word and hint, and Player 2 attempts to guess the word within a six-try limit. Unlike traditional implementations, this system supports **alphanumeric characters and symbols**, offering increased flexibility and a broader challenge.

By integrating features such as a music toggle, rule displays, and dynamic visual feedback, the project demonstrates advanced principles of **object-oriented programming (OOP)** and interactive software design. The result is a scalable, maintainable, and engaging educational tool that showcases the synergy between logic-heavy backend systems and responsive, user-centric frontends.

DESCRIPTION

This project consists of a **digitized, two-player implementation** of the classic Hangman game, themed around a "Wild West" narrative to enhance user engagement. Unlike standard single-player versions that rely on pre-defined word banks, this application introduces a **competitive "Bounty" mechanic** where Player 1 (the Lawman) inputs a secret word and a corresponding hint, while Player 2 (the Outlaw) attempts to solve the puzzle. The system is built using a **decoupled architecture**, strictly separating the game's core logic—such as guess tracking, win/loss detection, and input validation—from the graphical user interface. This separation ensures that the game engine functions independently of the visual presentation, allowing for easier debugging and future scalability.

From a user experience perspective, the application provides an **immersive multimedia environment** by integrating background music and dynamic image rendering that updates the hangman's state in real-time. The interface, developed using **Tkinter and BreezyPyGUI**, is designed to be fully responsive, ensuring that layout elements remain proportional regardless of window resizing. Furthermore, the system expands upon traditional gameplay by supporting **alphanumeric characters and symbols**, offering a more versatile and challenging experience. By combining **Object-Oriented Programming (OOP)** with event-driven GUI design, the project serves as a robust example of modular software development, blending creative storytelling with technical precision.

LIBRARIES & TOOLS

- **BreezyPyGUI**

This library serves as the high-level GUI framework used to define the main application window through the EasyFrame class. It simplifies the management of the window's grid system and provides a structured way to handle complex widget placements.

- **Tkinter**

As the underlying standard Python GUI toolkit, Tkinter is used for low-level operations such as creating a custom drawing Canvas and handling font styling. It is also responsible for binding user interactions, such as mouse clicks and physical keyboard presses, to specific game functions.

- **Pygame**

This library is integrated to provide multimedia support, specifically for managing the game's "Wild West" atmosphere through audio. It handles loading the background music file, looping it indefinitely during gameplay, and providing a toggle feature to pause or unpause the sound.

- **PIL (Pillow)**

The Python Imaging Library is used to handle all graphical assets, allowing the program to open and process background images. Its primary purpose is to enable dynamic resizing of these images using Resampling.LANCZOS, ensuring the visuals remain crisp even when the user changes the window size.

- **Random**

This standard Python module is utilized within the core logic to handle the game's element of chance. It is specifically used to select a secret word at random from the fallback list whenever a user does not provide a custom "bounty".

- **OS**

The OS module is employed for robust file management and error prevention. It checks for the existence of required asset paths, such as the music and background image files, ensuring the application handles missing files gracefully without crashing.

SYSTEM DESIGN & WORKFLOW

- **Initialization and Setup :** The workflow begins when the HangmanGUI class initializes the main window and loads multimedia assets, such as the background image via **PIL** and music via **Pygame**. The system starts in the MAIN_MENU state, offering options to play, view rules, or exit.
- **The "Bounty" Phase (Player 1 Input) :** When "Play Game" is selected, the application enters the setup phase where Player 1 acts as the "Lawman". The GUI collects a target word and a hint through screen entries. Upon clicking "Set Bounty," these inputs are passed to the HangmanGame class in the logic module, which validates the strings and prepares the game state.
- **Game Logic Engine (Backend) :** The HangmanGame class serves as the engine, managing the "hidden" state of the word. It handles alphanumeric character support and tracks the set of guessed letters. It calculates the current word state (revealing correct letters or underscores) and increments an error counter for every incorrect guess, up to a maximum of six.
- **Event-Driven Gameplay (Player 2 Action)** : The GUI switches to a split-screen view where Player 1's word is hidden by an opaque panel to prevent cheating. Player 2 interacts with an on-screen keyboard or physical keys. Each interaction triggers a "guess" event: the GUI calls the logic module to check the letter, and the logic module returns whether the move was valid or if the game has ended.
- **Dynamic Visual Rendering** : As Player 2 makes mistakes, the GUI's draw_hangman method reactively renders the cowboy's fate based on the backend error count. If the error count reaches six, the logic triggers a "Lost" state; if the word is completed, it triggers a "Won" state.
- **Termination and Reset :** Once the finished property in the logic module becomes true, the GUI displays a "Finished" screen with the final results. Users can then choose to set a "New Bounty," which resets the HangmanGame instance and returns the interface to the Player 1 setup screen.

IMPLEMENTATION

1.Backend Logic Implementation

The backend is encapsulated within the HangmanGame class, which serves as the central authority for the game state.

- **State Management:** The class initializes with a word list and a common clue, tracking the target word, player guesses using a Python set, and the error count.

- **Input Validation:** The guess method handles letter processing by converting inputs to uppercase and checking if the letter has already been guessed or if the game is already finished.

- **Dynamic Word Masking:** The current_state property uses a generator expression to build the display string, revealing correctly guessed letters while masking others with underscores.

- **Flexible Character Support:** The logic is implemented to automatically reveal non-alphanumeric characters (like spaces or punctuation) that are not part of the standard guessable keyboard, allowing for complex phrases.

2. Graphical User Interface Implementation

The frontend is implemented using an event-driven design, primarily utilizing the **Tkinter** and **BreezyPyGUI** frameworks.

- **State-Based Navigation:** The GUI manages different screens (Main Menu, How to Play, Game, and Finished) by updating a self.mode variable and re-drawing the canvas elements accordingly.

- **Two-Player "Bounty" Mechanic:** The interface uses a split-screen design where Player 1 enters a "Bounty" (word and hint) through tk.Entry widgets. Once set, an opaque panel is drawn over Player 1's side to hide the word from Player 2.

- **Responsive Visuals:** Implementation includes a <Configure> event binding on the canvas, which triggers the on_resize method. This method uses **PIL** to resize the background image to match the window dimensions dynamically.

- **Interactive Keyboard:** A virtual keyboard is programmatically generated on the canvas, where each button is linked to a lambda function that passes the specific character to the game logic.

3. Multimedia & Asset Integration

The implementation integrates external libraries to create an immersive "Wild West" atmosphere.

- **Audio Engine: Pygame.mixer** is initialized to load and loop background music. It includes a toggle system that tracks the music_on state to pause or unpause the audio stream.

- **Procedural Drawing:** The hangman figure is not a static image but is drawn procedurally using canvas.create_line and canvas.create_oval. The drawings are conditional, appearing only when the game.errors count reaches specific thresholds.

- **Resource Verification:** The code uses the os module to perform safety checks, ensuring assets like bgm.mp3 and background.png exist before attempting to load them, preventing runtime crashes.

RESULT & SCREENSHOTS

The implementation of the **Wild West Hangman** project delivers a functional, two-player experience by separating game logic from the graphical interface. The HangmanGame class manages state tracking, validating alphanumeric guesses and enforcing a six-error limit. A key result is the system's flexibility; it automatically reveals symbols like spaces or punctuation to ensure smooth gameplay.

On the frontend, **Tkinter** and **BreezyPyGUI** create a responsive interface that transitions between the menu, "Bounty" setup, and active play. The "Save the Cowboy" theme is rendered through procedural canvas drawing, showing body parts progressively as errors occur. Additionally, **PIL** dynamically resizes the background, while **Pygame** manages a toggleable, looped audio track.

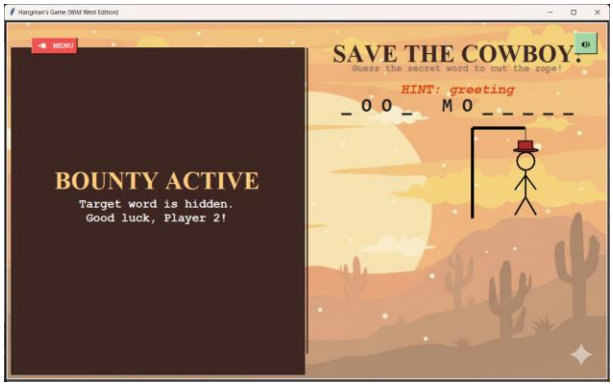
SCREENSHOTS

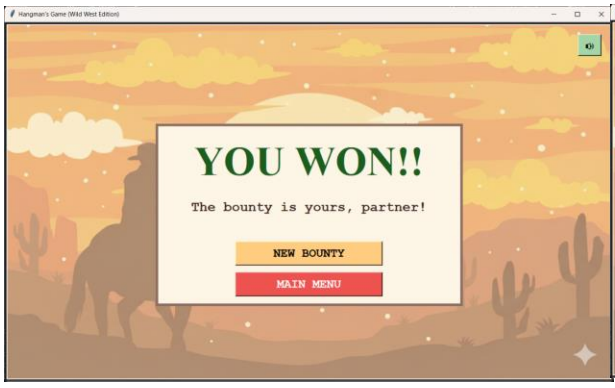
Hangman's Game (Wild West Edition)
MENU
SAVE THE COWBOY.
Guess the secret word to cut the rope!
HINT: greeting
_ _ _ _ _ _ _ _ _ _ _ _
BOUNTY ACTIVE
Target word is hidden.
Good luck, Player 2!

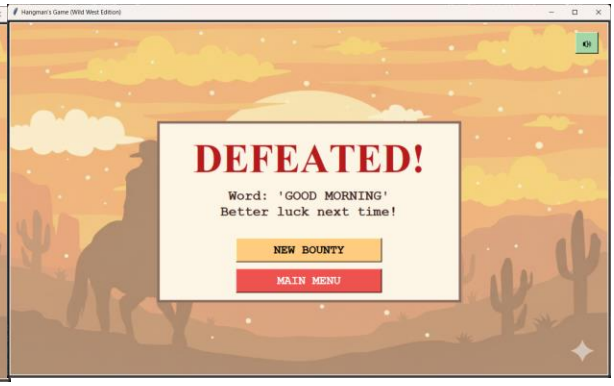
Hangman's Game (Wild West Edition)
MENU
SAVE THE COWBOY.
Guess the secret word to cut the rope!
HINT: greeting
_ _ _ _ _ M _ _ _ _ _ _
BOUNTY ACTIVE
Target word is hidden.
Good luck, Player 2!

Hangman's Game (Wild West Edition)
MENU
SAVE THE COWBOY.
Guess the secret word to cut the rope!
HINT: greeting
_ O O _ M O _ _ _ _ _
BOUNTY ACTIVE
Target word is hidden.
Good luck, Player 2!

Hangman's Game (Wild West Edition)
MENU
SAVE THE COWBOY.
Guess the secret word to cut the rope!
HINT: greeting
_ O O _ M O _ _ _ _ _
BOUNTY ACTIVE
Target word is hidden.
Good luck, Player 2!


Hangman's Game (Wild West Edition)
MENU
SAVE THE COWBOY.
Guess the secret word to cut the rope!
HINT: greeting
_ O O _ M O _ _ _ _ _
BOUNTY ACTIVE
Target word is hidden.
Good luck, Player 2!


Hangman's Game (Wild West Edition)
YOU WON!!
The bounty is yours, partner!
NEW BOUNTY
MAIN MENU


Hangman's Game (Wild West Edition)
DEFEATED!
Word: 'GOOD MORNING'
Better luck next time!
NEW BOUNTY
MAIN MENU

CONCLUSION & SCOPE

The **Wild West 2-Player Hangman** project successfully demonstrates the application of **Object-Oriented Programming (OOP)** and **event-driven GUI design** in Python. By separating the backend logic within the HangmanGame class from the **Tkinter** and **BreezyPyGUI** interface, the system achieves high maintainability and scalability. The integration of **Pygame** for audio and **PIL** for responsive visual rendering creates an immersive, multimedia-rich educational tool that transcends basic command-line versions. Ultimately, the project fulfills its objective of providing a polished, interactive, and flexible gaming environment for two players.

SCOPE

While the current version provides a robust local multiplayer experience, several enhancements could further evolve the project:

- **Online Multiplayer Support:** Implementing **socket programming** to allow players to set "bounties" and compete from different devices over a network.
- **Persistent Statistics:** Integrating a **database** (such as SQLite) to track player win/loss records and high scores over multiple sessions.
- **Difficulty Scaling:** Introducing a "Hard Mode" where the number of allowed errors is reduced or a countdown timer is added to increase the challenge.
- **Enhanced Animation:** Moving beyond static canvas drawings to include **frame-based animations** for the character and background transitions using **PIL**.
- **Global Word Bank:** Adding an optional single-player mode with a categorized library of words for solo practice.